

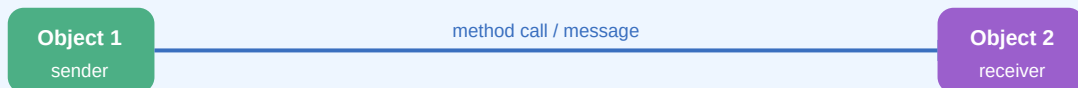
ICSE Class 9 • Computer Applications

Methods, Attributes & Encapsulation in Java

Unit 2 • Part 2



Message Passing



Attributes • Methods • Encapsulation

Message Passing • Getters & Setters • Data Hiding

NeverStop

Author: Sourabh Gorai

NeverStop Learning Platform

■ What are Attributes?

Attributes are **variables declared inside a class**. They store the **state** (data) of an object. Every object gets its own separate copy of all the attributes defined in its class.

■ Example – Student Class Attributes

```
class Student {  
    String name; // attribute – stores student's name  
    int rollNo; // attribute – stores roll number  
    int marks; // attribute – stores marks  
}
```

■ Each object of **Student** will have its own **name**, **rollNo**, and **marks** — completely independent from other objects.

■ What are Methods?

A **method** is a block of code inside a class that defines the **behavior** of an object — what it can do. Methods are called using the **dot operator** on an object.

■ Parts of a Method

Part	Example	Meaning
Return Type	void	Returns nothing
Method Name	display	Name of the method
Parameters	(String n, int m)	Input values passed in
Method Body	{ ... }	Actual code / logic

■ Method Example

```
class Student {  
    String name;  
    int marks;  
  
    void display() { // no parameter, no return  
        System.out.println("Name: " + name);  
        System.out.println("Marks: " + marks);  
    }  
}
```

■ Message Passing – Calling Methods

In OOP, when one object calls a method on another object, it is called **message passing**. The calling object sends a **message** (method name + arguments) and the receiving object executes it.

■ Three Forms of Message Passing

Type	Java Syntax	Example
No data, no return	<code>obj.method();</code>	<code>s1.display();</code>
With data (parameters)	<code>obj.method(val);</code>	<code>s1.setData("Aarav", 90);</code>
With return value	<code>type x = obj.method();</code>	<code>int r = calc.add(5,3);</code>

■ Complete Message Passing Example

```
class Student {
    String name; int marks;

    void setData(String n, int m) { // receives message with data
        name = n; marks = m;
    }
    void display() { // receives message, no data
        System.out.println(name + " scored " + marks);
    }
}
class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setData("Priya", 88); // message with data
        s1.display(); // simple message
    }
}
```

Output: Priya scored 88

■ Method with Return Value

```
class Calculator {
    int add(int a, int b) { // returns int
        return a + b;
    }
}
class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result = calc.add(5, 3); // message passing
        System.out.println("Sum = " + result);
    }
}
```

```
}
```

Output: Sum = 8

■ Encapsulation – Data Hiding

Encapsulation means wrapping the **data (attributes)** and **methods** together inside a class, and **hiding the data** from direct outside access. Think of a medicine capsule ■ — everything is packed inside, you cannot touch what's inside directly.

■ Two Parts of Encapsulation

Part	How	Purpose
Data Hiding	Use private keyword	Block direct access from outside
Controlled Access	Use public getters/setters	Allow safe, validated access

■ Encapsulation – BankAccount Example

```
class BankAccount {
    private double balance; // private – no direct outside access

    public void deposit(double amount) { // setter-style method
        if (amount > 0) balance += amount;
    }

    public double getBalance() { // getter method
        return balance;
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(5000);
        System.out.println("Balance: " + acc.getBalance());
        // acc.balance = -9999; ERROR! private – not allowed
    }
}
```

Output: Balance: 5000.0

■ Complete Example – Encapsulation + Getters/Setters

```
class Student {
    private String name;
    private int marks;

    public void setName(String n) { name = n; }
    public void setMarks(int m) {
```

```

        if (m >= 0 && m <= 100) marks = m;
        else System.out.println("Invalid marks!");
    }
    public String getName() { return name; }
    public int getMarks() { return marks; }

    public void display() {
        System.out.println("Name: " + name + " | Marks: " + marks);
    }
}
class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setName("Rohan");
        s1.setMarks(95);
        s1.display();
        s1.setMarks(150); // invalid – will print error
    }
}

```

Output:

```

Name: Rohan | Marks: 95
Invalid marks!

```

■ Access Modifiers – Quick Note

Modifier	Who Can Access?
public	Everyone — from any class, any package
private	Only inside the same class — no one else
default (none)	Classes in the same package only

■ Quick Revision Summary

Term	Meaning
Attribute	Variable inside a class — stores the object's state/data
Method	Function inside a class — defines the object's behavior
Message Passing	Calling a method on an object using the dot operator
Encapsulation	Bundling data + methods together; hiding data using private
private	Access modifier — restricts access to the same class only
public	Access modifier — accessible from anywhere

Getter	Public method that returns the value of a private attribute
Setter	Public method that sets/validates a private attribute's value

■ Memory Trick – A.M.E.

A → **Attributes** = Aadhaar card data ■ (your info / state)

M → **Methods** = Machines in a factory ■■ (what gets done)

E → **Encapsulation** = Envelope sealed ■ (hidden inside, safe)

Remember: **private** = locked inside ■ | **public** = open door ■

⇒ ■ Practice Questions

1. What is the difference between an attribute and a method?
2. What is message passing in Java? Give an example.
3. Write a class **Rectangle** with attributes length and breadth, and a method **area()** that returns the area.
4. What is encapsulation? Why is it important in OOP?
5. What is the difference between **private** and **public** access modifiers?
6. Write getter and setter methods for a class **Employee** with private attribute **salary**.
7. What will happen if you try to access a private variable directly from outside its class?
8. Write a method **isPass()** that returns **true** if marks ≥ 35 , otherwise **false**.

■■ Important Tips for Exams

- ✓ Always declare attributes as **private** for proper encapsulation.
- ✓ Getter method name convention: **getVariableName()** — e.g., getName(), getMarks().
- ✓ Setter method name convention: **setVariableName(val)** — e.g., setName(String n).
- ✓ A method with **void** return type does not use the return statement (or uses bare return;).
- ✓ Message passing = calling a method on an object — always uses the **dot operator**.
- ✓ Encapsulation improves **security, maintainability**, and prevents invalid data.
- ✓ In ICSE exams, always write access modifiers (**public/private**) before methods and attributes.

